Experiment No.08

Aim: Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR).

Theory:

1. Dataset Source

Dataset: Fashion-MNIST Dataset

Source link:
https://github.com/zalandoresearch/fashion-mnist

2. Dataset Description

Fashion-MNIST is a widely used image dataset consisting of grayscale images of clothing items.

Features:

- Each sample is a 28×28 pixel grayscale image
- Pixel values range from 0 to 255

Target Variable:

- Multi-class label (0–9 representing clothing categories)

Classes:

- T-shirt/top
- Trouser
- Pullover
- Dress
- Coat
- Sandal
- Shirt
- Sneaker
- Bag
- Ankle boot

3. Mathematical Formulation of CNN

Convolution Operation

$$S(i,j) = (X * K)(i,j) = \sum_m \sum_n X(i+m, j+n) \cdot K(m,n)$$

Activation Function (ReLU)

$$f(x) = \max(0, x)$$

Pooling Operation

Reduces spatial dimensions:

P=max(X)

Softmax Function (Output Layer)

$$P(y=i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Loss Function (Categorical Cross-Entropy)

$$L = -\sum y_i \log(p_i)$$

Optimization

Weights updated using Adam optimizer:

$$w = w - \eta \frac{\partial L}{\partial w}$$

4. Algorithm Limitations

- Requires high computational resources (GPU preferred)
- Large training data needed for better performance
- Prone to overfitting without regularization
- Difficult to interpret internal representations
- Performance sensitive to hyperparameters
- Struggles with very small or noisy datasets

5. Methodology / Workflow

Steps:

1. Load Fashion-MNIST dataset
2. Normalize pixel values (0–1 range)
3. Reshape images for CNN input
4. Convert labels using one-hot encoding
5. Build CNN architecture:
    - Convolution layers
    - Pooling layers
    - Fully connected layers
6. Compile model using:
    - Adam optimizer
    - Categorical cross-entropy
7. Train model using training dataset
8. Validate using validation split
9. Evaluate on test dataset
10. Generate predictions
11. Compute confusion matrix and classification report
12. Plot accuracy graph

6. Performance Analysis

Evaluation Metrics Used:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

Interpretation:

- CNN effectively captures spatial patterns in images
- Deeper layers improve feature extraction
- Performance is influenced by dataset complexity

7. Hyperparameter Tuning

Parameters Tuned:

Parameter	Values Tested
Filters	16, 32, 64
Kernel Size	(3×3)
Epochs	5–10
Batch Size	32, 64
Dropout	0.3 – 0.5

Final Configuration:

- Conv Layers: 2
- Filters: 16, 32
- Epochs: 8
- Batch Size: 64
- Dropout: 0.4

Impact of Tuning:

- Improved accuracy and stability
- Reduced overfitting
- Faster convergence with Adam optimizer
- Better generalization on test data

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical

from sklearn.metrics import confusion_matrix, classification_report

# Load Dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)
# Class Labels
labels = ['Top','Trouser','Pullover','Dress','Coat',
        'Sandal','Shirt','Sneaker','Bag','Boot']
# Show Sample Images (DIFFERENT STYLE)
plt.figure(figsize=(10,6))
for i in range(12):
    plt.subplot(3,4,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(labels[y_train[i]])
    plt.axis('off')
plt.suptitle("Sample Fashion Items", fontsize=14)
plt.tight_layout()
plt.show()
# Preprocessing
X_train = X_train / 255.0
X_test = X_test / 255.0
# Reshape (important for CNN)
X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)
# One-hot encoding
y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
```

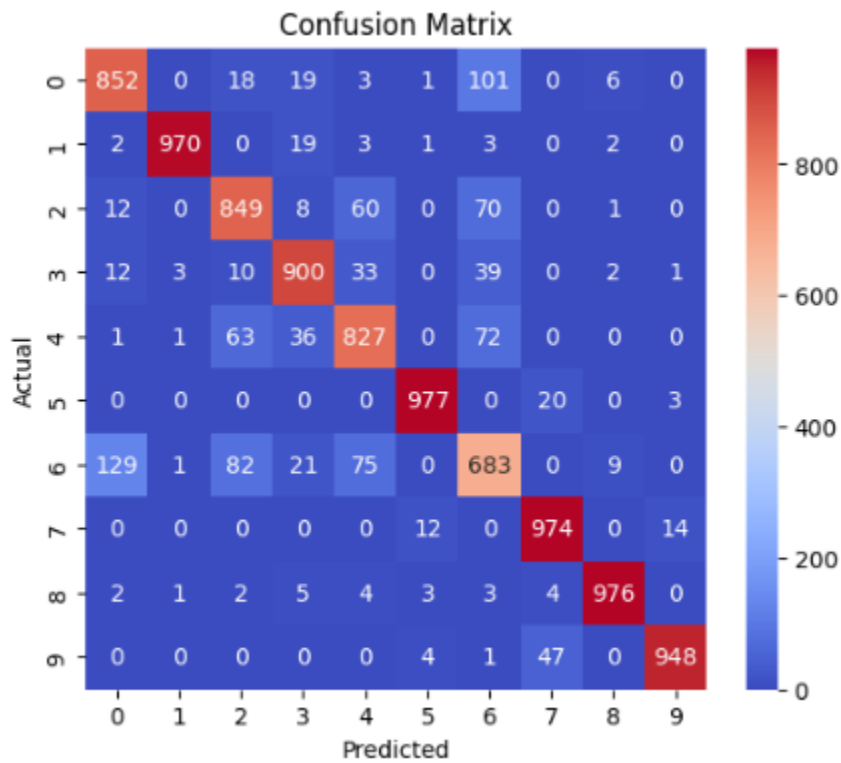
```
# CNN Model (DIFFERENT ARCHITECTURE)
model = Sequential()
model.add(Conv2D(16, (3,3), activation='relu', input_shape=(28,28,1)))
model.add(MaxPooling2D(2,2))
model.add(Conv2D(32, (3,3), activation='relu'))
model.add(MaxPooling2D(2,2))
model.add(Flatten())
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.4))
model.add(Dense(10, activation='softmax'))
# Compile
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train Model
history = model.fit(
    X_train, y_train,
    epochs=8,
    batch_size=64,
    validation_split=0.2
)
# Evaluate
loss, acc = model.evaluate(X_test, y_test)
print("Test Accuracy:", acc)
# Predictions
y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true = np.argmax(y_test, axis=1)
# Confusion Matrix
cm = confusion_matrix(y_true, y_pred_classes)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='coolwarm')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
# Classification Report
```

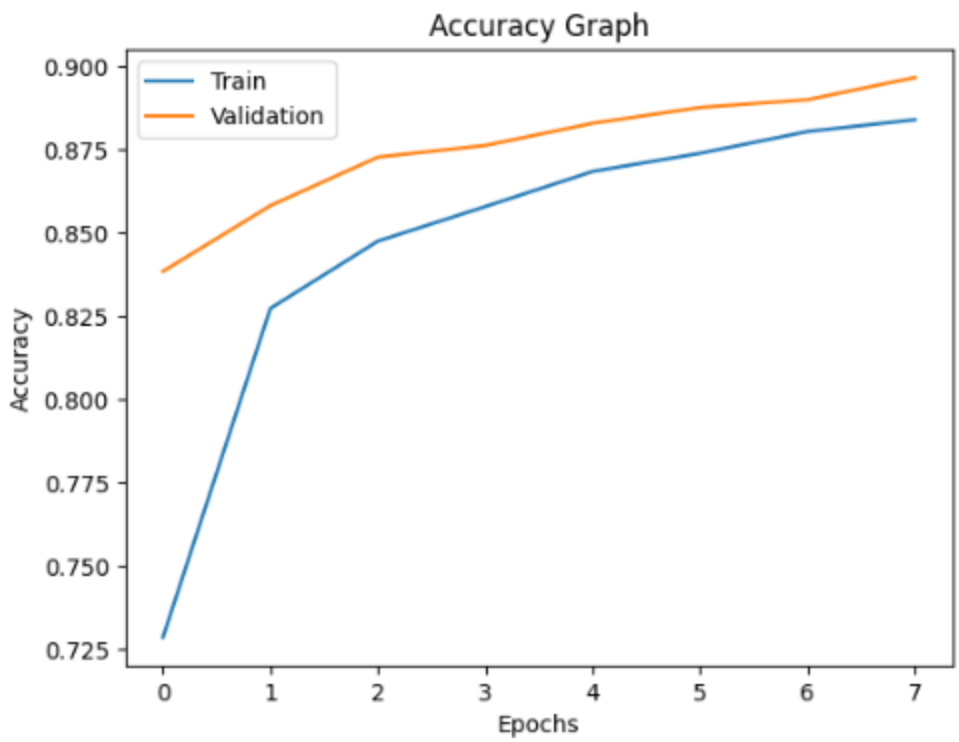
```
print(classification_report(y_true, y_pred_classes))
# Accuracy Graph
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Validation')
plt.title("Accuracy Graph")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 1us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training shape: (60000, 28, 28)
Testing shape: (10000, 28, 28)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_sh
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/8
750/750 ━━━━━━━━━━━━━━━━━━━━ 32s 41ms/step - accuracy: 0.7286 - loss: 0.7493 - val_accuracy: 0.8385 - val_loss: 0.4414
Epoch 2/8
750/750 ━━━━━━━━━━━━━━━━━━━━ 24s 32ms/step - accuracy: 0.8274 - loss: 0.4831 - val_accuracy: 0.8583 - val_loss: 0.3860
Epoch 3/8
750/750 ━━━━━━━━━━━━━━━━━━━━ 23s 31ms/step - accuracy: 0.8476 - loss: 0.4279 - val_accuracy: 0.8728 - val_loss: 0.3513
Epoch 4/8
750/750 ━━━━━━━━━━━━━━━━━━━━ 25s 33ms/step - accuracy: 0.8580 - loss: 0.3948 - val_accuracy: 0.8763 - val_loss: 0.3336
Epoch 5/8
750/750 ━━━━━━━━━━━━━━━━━━━━ 24s 32ms/step - accuracy: 0.8685 - loss: 0.3700 - val_accuracy: 0.8830 - val_loss: 0.3131
Epoch 6/8
750/750 ━━━━━━━━━━━━━━━━━━━━ 24s 32ms/step - accuracy: 0.8740 - loss: 0.3498 - val_accuracy: 0.8878 - val_loss: 0.3051
Epoch 7/8
750/750 ━━━━━━━━━━━━━━━━━━━━ 24s 31ms/step - accuracy: 0.8805 - loss: 0.3348 - val_accuracy: 0.8901 - val_loss: 0.2978
Epoch 8/8
750/750 ━━━━━━━━━━━━━━━━━━━━ 42s 33ms/step - accuracy: 0.8841 - loss: 0.3210 - val_accuracy: 0.8967 - val_loss: 0.2866
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.8956 - loss: 0.2994
Test Accuracy: 0.8956000208854675
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step
```





Conclusion:

The Convolutional Neural Network successfully classified fashion images with high accuracy by learning hierarchical visual features. Compared to simpler models, CNN demonstrated superior performance in handling image data. Proper preprocessing, architecture design, and hyperparameter tuning significantly enhanced model effectiveness, making CNN a powerful approach for image classification tasks.